SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 40%

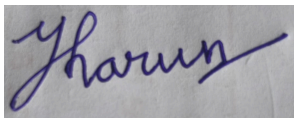
Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**
Question Count: 5

Code of Conduct

I Tharunkumar.S / 192511416 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Analytical Problem Solving

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.
2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.
3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.
4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.
5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Selection rationale and comparison

Original assessment question (verbatim): 1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.

Problem interpretation and decision criteria

For a mission-critical cloud datacenter, the hypervisor is a foundational isolation and resource-scheduling layer. The selection must satisfy predictable application latency, strong separation of tenants, coordinated maintenance, high availability and auditable operations. “Mission critical” means service interruption and data leakage have high consequences; choice cannot be based only on installation convenience.

Type-1 (bare-metal) hypervisors execute directly on physical hardware and host multiple guest operating systems. Examples include VMware ESXi and bare-metal Hyper-V deployments. Type-2 (hosted) hypervisors, such as VMware Workstation and VirtualBox, run above a general-purpose host operating system. Actual performance depends on drivers, hardware-assisted virtualization, workload and configuration.

Full comparison

Dimension	Type-1 bare-metal	Type-2 hosted
Execution layer	Directly above server hardware	Runs on a host operating system
Production performance	Designed for server resource control	Extra host OS scheduling and services
Security exposure	Hypervisor + management plane	Hypervisor + full host OS ecosystem
HA / live migration	Often available on enterprise platforms	Generally limited for datacenter clustering
Administration	Central policy and fleet management	Usually desktop- or user-focused
Best fit	Production cloud workloads	Lab, development, individual workstations

Decision criteria applied to the scenario

Prioritize blast-radius containment, supported live migration and clustering, centralized policy, memory and CPU resource controls, firmware and host lifecycle management, and recovery objectives. Evaluate workload benchmarks and operational tooling for the intended vendor rather than assuming every Type-1 product provides identical capabilities.

For production, select a supported Type-1 platform with redundant hosts, separated management plane, hardened configurations and automated recovery. Use Type-2 virtualization for education, prototyping and desktop testing where convenience matters more than dedicated infrastructure controls.

Performance, security and operations

Question in focus: 1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.

Full comparison

Engineering controls and why they matter

Control	Reason
Dedicated management network	Limits administrative exposure
HA spare capacity	Supports recovery after a host failure
Golden image + patch policy	Improves consistency and remediation
Workload SLO monitoring	Detects service-level degradation

High-availability architecture

Question in focus: 1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.

Technical design / conceptual architecture

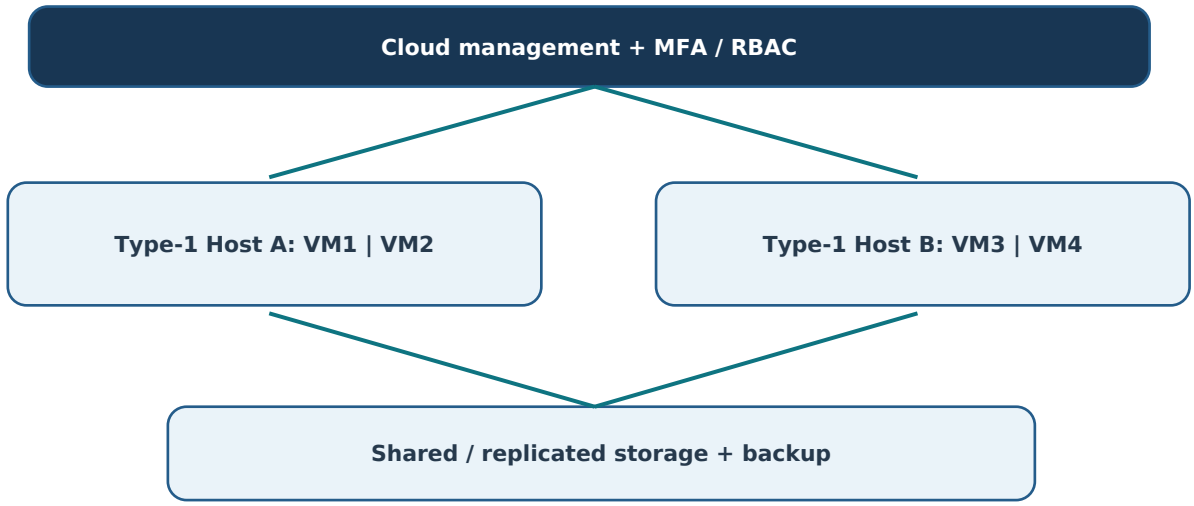


Figure: Analytical reference architecture illustrating the key software, hardware, trust or performance dependencies for this scenario. Connections represent principal control or data paths, not a vendor-specific wiring plan.

Decision criteria applied to the scenario

Prioritize blast-radius containment, supported live migration and clustering, centralized policy, memory and CPU resource controls, firmware and host lifecycle management, and recovery objectives. Evaluate workload benchmarks and operational tooling for the intended vendor rather than assuming every Type-1 product provides identical capabilities.

For production, select a supported Type-1 platform with redundant hosts, separated management plane, hardened configurations and automated recovery. Use Type-2 virtualization for education, prototyping and desktop testing where convenience matters more than dedicated infrastructure controls.

Design verification notes

Document ownership of each cluster, network and datastore. Confirm redundant failure domains and migration compatibility. Run a restore, not merely a backup, to establish recoverability.

Testing plan and final recommendation

Question in focus: 1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.

Performance, security and manageability analysis

Performance: in a bare-metal design, the hypervisor schedules virtual CPUs and brokers guest memory and I/O directly against the hardware. Hardware virtualization features, paravirtualized drivers and virtual-machine placement policies can reduce overhead. Under either architecture, CPU ready time, memory pressure, NUMA placement and shared-storage contention determine real service performance.

Security: both architectures require hypervisor updates, guest hardening and network segmentation. A hosted design additionally relies on the security and patch state of the host desktop OS and its broader application ecosystem. A Type-1 deployment usually offers a smaller dedicated host footprint, but an exploited management plane or hypervisor still threatens multiple guests.

Manageability: datacenter platforms typically expose central inventory, role-based access, templates, backup integration, health monitoring, automated placement and non-disruptive migration. Hosted tools favor per-user convenience and are weaker choices for clustered enterprise operation. HA features are product-dependent and must be verified.

Assessment response: method, evidence and acceptance

Use change-controlled tests and document the actual measured results. A passing infrastructure-level metric does not replace application-level reliability testing.

Multi-tenant isolation objectives

Original assessment question (verbatim): 2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

Scenario interpretation and security objectives

Finance, healthcare and student portals have distinct data sensitivity and regulatory responsibilities. Running them on shared physical hosts offers better utilization but introduces risks from co-residency, management compromise, misconfigured virtual networks and resource interference. The required outcome is controlled sharing of hardware without uncontrolled sharing of trust.

Isolation dimensions are compute, memory, storage, network, identity, management and operational visibility. A VM boundary is important, but not sufficient without virtual network separation, encrypted data paths and storage access controls.

Trust-boundary inventory

Boundary	Enforcement example	Primary validation
Compute/memory	Hypervisor guest isolation, quotas	Try unauthorized cross-VM access
Virtual network	Tenant VLAN/overlay + microsegmentation	Deny cross-tenant routes
Storage	Virtual disk ACLs + encryption keys	Reject unauthorized disk access
Identity/admin	Scoped RBAC, MFA, audit logs	Least-privilege tests
Operations	Independent backup/log partitions	Restore/access audit

Tenant-aware design decisions

Use separate tenant-specific VMs or groups, dedicated virtual networks and distributed firewall policies. Administrators receive least-privilege permissions limited to their tenant resources; high-impact host tasks require stronger approvals and auditing. Different data classes should have independent encryption keys and backup retention policies.

The shared platform must offer resource reservations and caps to prevent noisy-neighbor disruption. High-sensitivity workloads may require dedicated host pools where legal or risk assessments demand stronger physical separation.

Isolation mechanisms in depth

Question in focus: 2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

Trust-boundary inventory

Engineering controls and why they matter

Control	Reason
Tenant-level key ownership	Reduces cross-tenant data exposure
Virtual network default deny	Prevents unauthorized east-west traffic
Separate admin roles	Limits management blast radius
Resource reservations	Reduce noisy-neighbor risk

Attack surfaces and mitigations

Question in focus: 2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

Technical design / conceptual architecture

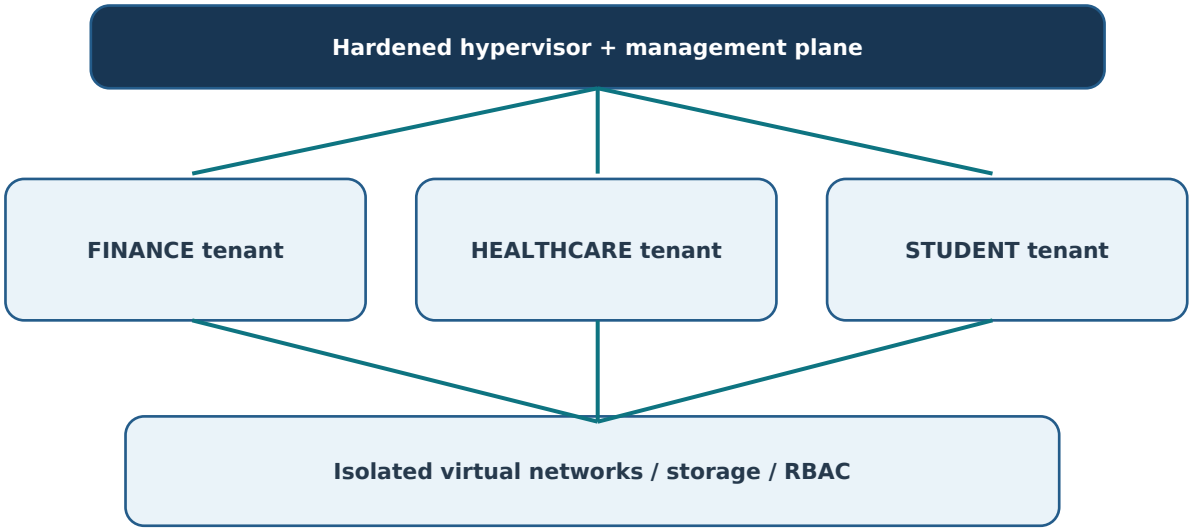


Figure: Analytical reference architecture illustrating the key software, hardware, trust or performance dependencies for this scenario. Connections represent principal control or data paths, not a vendor-specific wiring plan.

Tenant-aware design decisions

Use separate tenant-specific VMs or groups, dedicated virtual networks and distributed firewall policies. Administrators receive least-privilege permissions limited to their tenant resources; high-impact host tasks require stronger approvals and auditing. Different data classes should have independent encryption keys and backup retention policies.

The shared platform must offer resource reservations and caps to prevent noisy-neighbor disruption. High-sensitivity workloads may require dedicated host pools where legal or risk assessments demand stronger physical separation.

Design verification notes

Enforce policy at more than one layer: hypervisor access, virtual switching, host firewalls, key management and administrator identity. Assume one layer may fail.

Security testing and conclusions

Question in focus: 2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

How the virtualization architecture enforces separation

The hypervisor mediates guest access to CPU, memory, devices and privileged instructions using hardware features such as Intel VT-x/AMD-V and second-level address translation. Virtual disks attach to specified guests under storage policies. Virtual switches and overlays can isolate traffic even when physical NICs are shared.

Microsegmentation permits only authorized service-to-service communication: for example, healthcare front ends may reach healthcare application and database services, not financial data services. Private management NICs, bastion access, multi-factor authentication and signed configuration changes reduce administration risk.

Memory and CPU quotas protect availability but cannot eliminate every microarchitectural side channel. Sensitive environments may disable unsafe device sharing, adopt virtual TPM and measured boot, encrypt disks and enforce host patch SLAs.

Assessment response: method, evidence and acceptance

Treat attempted unauthorized tenant access as a failed test only when the access is denied and the event is recorded; ensure legitimate access still works.

Host snapshot and triage

Original assessment question (verbatim): 3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

Snapshot interpretation and assumptions

The given values - CPU 92%, memory 88%, storage I/O latency 35 ms, and mean boot time 170 seconds - indicate an overloaded or poorly provisioned host. However, a single snapshot cannot establish whether the load is sustained, whether storage latency is read or write latency, or where boot time is spent. Correlate at least several peak and nonpeak intervals before making destructive changes.

High CPU utilization may imply guest CPU contention; high RAM allocation can trigger reclaiming or swapping; 35 ms I/O latency is a strong reason to investigate the shared storage path; slow boot may be caused by storage queues, image distribution, initialization scripts, DHCP/DNS or a saturated host.

Quantitative triage

Metric	Observed	Interpretation / first check
Host CPU	92%	Check sustained demand, CPU ready, oversizing
Host memory	88%	Inspect active memory, ballooning and swap
Storage I/O latency	35 ms	Investigate queue depth and datastore health
Average VM boot time	170 s	Split startup phases; find dominant delay

Likely root-cause chain

If too many VMs start together, CPU demand increases while boot disk reads and writes queue on shared storage. As memory pressure rises, guest or host paging can add more disk I/O. This amplifies storage latency and extends boot times. Another possibility is an unhealthy storage backend or network path causing a primary storage bottleneck independent of CPU.

Do not immediately attribute a 170-second boot to the hypervisor alone; use timestamps for VM power-on, firmware/OS startup, cloud-init, networking and application readiness to identify the dominant phase.

Root-cause investigation

Question in focus: 3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

Quantitative triage

Engineering controls and why they matter

Control	Evidence
CPU ready / steal	Shows whether guest vCPUs wait to run
Ballooning and swapping	Quantifies active memory distress
I/O queue depth + p95 latency	Differentiates saturation from isolated spikes
Boot phase event timestamps	Locates slow provisioning phase

Corrective action architecture

Question in focus: 3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

Technical design / conceptual architecture

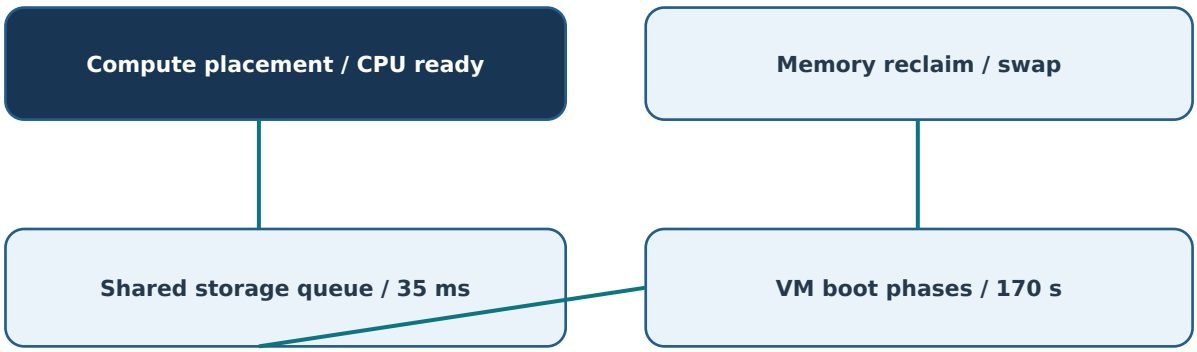


Figure: Analytical reference architecture illustrating the key software, hardware, trust or performance dependencies for this scenario. Connections represent principal control or data paths, not a vendor-specific wiring plan.

Likely root-cause chain

If too many VMs start together, CPU demand increases while boot disk reads and writes queue on shared storage. As memory pressure rises, guest or host paging can add more disk I/O. This amplifies storage latency and extends boot times. Another possibility is an unhealthy storage backend or network path causing a primary storage bottleneck independent of CPU.

Do not immediately attribute a 170-second boot to the hypervisor alone; use timestamps for VM power-on, firmware/OS startup, cloud-init, networking and application readiness to identify the dominant phase.

Design verification notes

Correlation, not one snapshot, establishes causality. Align all utilization and event metrics on a synchronized timeline to avoid false diagnosis.

Monitoring and re-validation

Question in focus: 3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

Detailed bottleneck analysis and evidence collection

CPU: collect per-host utilization over time, CPU ready/steal, core oversubscription, VM vCPU allocation, throttling and NUMA topology. A 92% reading near a sustained peak leaves little headroom for bursts or host failure. Reduce oversized vCPU assignments and migrate workloads; add capacity if demand persists.

Memory: inspect actual active/consumed RAM, ballooning, swapping, page faults and host reclamation. The 88% figure does not alone prove thrashing; it is a warning signal. Increase RAM, move VMs, correct reservations or reduce memory-heavy services based on observed paging.

Storage: measure p95/p99 latency, queue depth, IOPS, throughput, read/write balance, datastore free space, thin-provisioning health and backend disk/network saturation. At 35 ms, review noisy neighbors, snapshots, insufficient cache, overloaded HDD tiers and storage fabric bottlenecks.

Boot: break the 170-second figure into VM scheduling, disk preparation, OS initialization and application readiness; correlate with storage latency and simultaneous start requests.

Assessment response: method, evidence and acceptance

Before-and-after tests must use comparable VM mixes and demand. Otherwise a lower utilization figure may simply reflect reduced business load.

SDDC principles and comparison

Original assessment question (verbatim): 4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

Traditional vs software-defined datacenter (SDDC)

A traditional datacenter often provisions servers, storage volumes and network connectivity through separate device-specific workflows. Software Defined Datacenter (SDDC) abstracts compute, storage and networking into software-controlled resource pools exposed through policy and APIs. The physical equipment remains necessary, but provisioning decisions can be automated.

The agility benefit is primarily faster, repeatable provisioning and centralized policy management - not an automatic guarantee of lower latency or cost. Poorly designed automation can reproduce a bad configuration at scale.

Capability comparison

Capability	Traditional approach	Software-defined approach
Compute	Manual server and OS requests	Template/API-driven VM orchestration
Storage	Per-array/LUN provisioning	Policy-based volume creation
Networking	Switch/router port configuration	Logical segments + SDN rules
Changes	Separate change tickets	Versioned IaC / coordinated workflows
Elasticity	Slow procurement/configuration	Fast scaling within physical limits

SDDC foundations

Compute virtualization provides VM templates, placement, quotas, scheduling and lifecycle automation. Software-defined storage pools capacity and performance classes while exposing replication, snapshots and policy-based provisioning. Software-defined networking (SDN) creates logical networks, virtual routing, security groups and overlays independent of individual switch ports.

An orchestration layer links all three domains so a single approved request can deploy an application with CPU/RAM, volume, subnet, firewall policies and monitoring configuration consistently.

Cross-layer virtualization

Question in focus: 4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

Capability comparison

Engineering controls and why they matter

Layer	Virtualized resource
Compute	VMs, templates, schedulers
Storage	Volumes, snapshots, policy classes
Network	Overlays, virtual routing and firewall
Orchestration	End-to-end deployments and audit

Provisioning reference architecture

Question in focus: 4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

Technical design / conceptual architecture

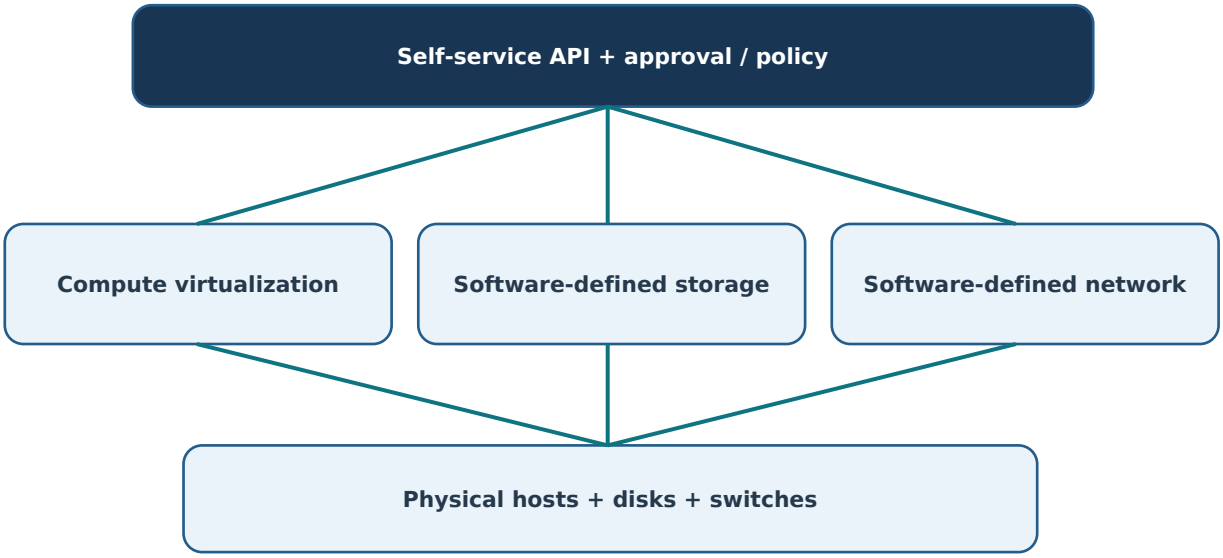


Figure: Analytical reference architecture illustrating the key software, hardware, trust or performance dependencies for this scenario. Connections represent principal control or data paths, not a vendor-specific wiring plan.

SDDC foundations

Compute virtualization provides VM templates, placement, quotas, scheduling and lifecycle automation. Software-defined storage pools capacity and performance classes while exposing replication, snapshots and policy-based provisioning. Software-defined networking (SDN) creates logical networks, virtual routing, security groups and overlays independent of individual switch ports.

An orchestration layer links all three domains so a single approved request can deploy an application with CPU/RAM, volume, subnet, firewall policies and monitoring configuration consistently.

Design verification notes

Declarative orchestration needs change review, rollback capability and drift checks. Each deployed resource must map to an approved policy and a physical capacity budget.

Agility evaluation and tradeoffs

Question in focus: 4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

Compute, storage and network virtualization

Compute: hypervisor clusters consolidate workloads and deliver rapid VM provisioning from standardized golden images. Scheduling policies account for affinity, availability zones, reservations and resource contention. Infrastructure-as-code turns configuration into version-controlled templates.

Storage: software-defined storage abstracts local and shared devices, allocates volumes on demand, enforces durability/service classes and can replicate data across failure domains. Storage policies can offer encrypted volumes and automatic placement; latency remains bounded by physical media and network capacity.

Network: SDN centralizes logical network intent. Virtual switches, VXLAN/Geneve-like overlays, virtual routers and distributed firewall rules let teams create isolated application networks without re-cabling. Routing and segmentation must be observable and tested to prevent policy drift.

Cross-layer orchestration attaches the right VM to the right datastore and logical network using a single request, enabling rapid scale-out or teardown with reproducible audit trails.

Assessment response: method, evidence and acceptance

Capture before-and-after provisioning time and policy compliance. Benefits arise from repeatable workflows rather than virtualization by itself.

Federation failure analysis

Original assessment question (verbatim): 5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Problem interpretation: federation identity mismatch

Multi-cloud federation permits users and workloads to access services across independent cloud domains using trusted identity assertions or tokens. Failure occurs when providers disagree on the meaning of subject identifiers, email aliases, issuer names, audiences, group claims, login assurance, token lifetime or entitlement mappings.

The central design principle is to authenticate once at an authoritative identity provider while each resource provider verifies provenance and applies its own least-privilege authorization. Federation solves inter-domain trust; it does not remove the need for per-service access control.

Common mismatch patterns

Mismatch	Likely failure	Correction
Email used as key	Alias/collision creates wrong account link	Key on issuer + immutable subject
Audience mismatch	Token refused by provider	Per-provider audience config
Group names differ	Incorrect permissions	Versioned entitlement mapping
Stale signing key	Validation failures	Automated JWKS rotation/testing
Clock skew/expiry	Intermittent login errors	Time sync and bounded tolerance

Identity standards in the design

OpenID Connect (OIDC) adds identity assertions to OAuth 2.0 and is well suited to interactive web/mobile sessions. SAML 2.0 remains relevant for enterprise SSO. OAuth 2.0 scopes and audience-bound tokens support delegated API access; a workload identity federation design avoids long-lived cloud API secrets.

Use issuer + stable subject as the identity key. Email addresses can change and are not sufficient as unique security identifiers. Normalize required attributes and version claim mappings; define the authoritative source for roles, organization memberships and assurance levels.

Federated identity assurance

Question in focus: 5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Common mismatch patterns

Engineering controls and why they matter

Security control	Purpose
Stable issuer + subject ID	Prevents account-collision linking
Audience-bound short-lived token	Limits token reuse
MFA + key rotation	Improves assurance and signature trust
Local authorization + logs	Enforces scoped access

Secure identity trust architecture

Question in focus: 5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Technical design / conceptual architecture

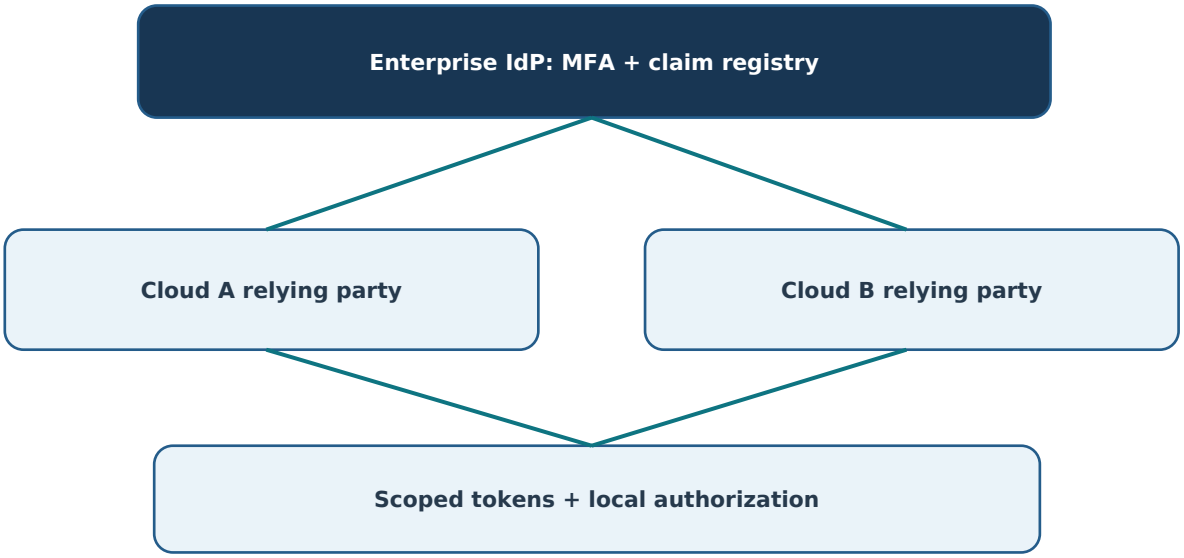


Figure: Analytical reference architecture illustrating the key software, hardware, trust or performance dependencies for this scenario. Connections represent principal control or data paths, not a vendor-specific wiring plan.

Identity standards in the design

OpenID Connect (OIDC) adds identity assertions to OAuth 2.0 and is well suited to interactive web/mobile sessions. SAML 2.0 remains relevant for enterprise SSO. OAuth 2.0 scopes and audience-bound tokens support delegated API access; a workload identity federation design avoids long-lived cloud API secrets.

Use issuer + stable subject as the identity key. Email addresses can change and are not sufficient as unique security identifiers. Normalize required attributes and version claim mappings; define the authoritative source for roles, organization memberships and assurance levels.

Design verification notes

The IdP proves identity, while each relying party enforces its own access policy. Every token is validated for sender, intended audience, expiry and integrity.

Privacy, tests and resolution

Question in focus: 5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Secure reference architecture and trust flow

An authoritative enterprise IdP authenticates people using phishing-resistant MFA for sensitive functions. Trust agreements register each cloud service provider, permitted client IDs, audience values, signing keys, encryption settings, clock-skew policy and minimum assurance requirements.

Upon login, the relying party redirects to the IdP and verifies returned signatures, issuer, audience, nonce, expiry and state; it then maps stable subject and attributes to a local account or JIT-provisioned identity. Authorization uses role or attribute policies and explicit deny. APIs verify access tokens independently of user-facing ID tokens.

For service-to-service tasks, employ short-lived federated workload tokens tied to a specific workload and resource audience rather than embedded credentials. All trust configuration changes receive review and audit.

Assessment response: method, evidence and acceptance

Test account lifecycle and mismatched attributes in both directions. A system that logs in correctly but grants excess permissions remains insecure.

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow

Total Marks Awarded: